website page at https://mng.bz/qRq6 provides a long list of supported vector stores. For simplicity, we will use the `MemoryVectorStore`, which uses an in-memory, ephemeral vector store and does not need additional configuration to employ.

Vector stores serve two primary functions: they ingest and encode documents into a searchable format, and they perform semantic searches to retrieve relevant documents based on their meaning. Let's see how to create and use a `MemoryVectorStore` instance to store documents and then perform a similarity search with it. The following listing shows the two key aspects of information retrieval: embedding generation (with `OpenAIEmbeddings`) and vector storage (via the `MemoryVectorStore.fromTexts` call).

Listing 6.6 Performing similarity search

```
MemoryVectorStore.fromTexts(
    ["Hello world", "Bye bye", "hello nice world"],
    [{ id: 2 }, { id: 1 }, { id: 3 }],
    new OpenAIEmbeddings({apiKey})
);

console.log(
await vectorStore.similaritySearch("hello world", 1));

/* Example output:
[
Document {
   pageContent: "Hello world",
   metadata: { id: 2 }
}
]
*/
```

List of document texts to be stored

Array of metadata objects corresponding to each text

Instance of OpenAI embeddings generator for vector conversion

Searches for documents similar to "hello world", limiting to one result

Once you have the list of documents stored in a vector store that is suitable for similarity search, you can query the database (via prompting) to answer specific questions or retrieve relevant information based on the context provided. This capability allows you to use the power of semantic search to find precise answers quickly and efficiently. This is the job of the `Retrieval` objects in LangChain.

6.2.3 Document retrieval

Document retrieval is the process of finding and returning the most relevant documents from our vector store based on a given query. Given an instance of a vector storage class, we can ask for the `Retriever` instance (using the `asRetriever` method).

The retrievers allow us to chain the process of retrieving the relevant documents with LangChain in a composable way and let us create robust and flexible query pipelines. The following listing shows how to create the `Retriever` instance and to include it in a prompt chain that performs a chat-like experience regarding the information included in the documents.